

Reforme Studio Pilates — Contexto del proyecto

Documento de contexto versionado. Objetivo: que cualquier persona (o agente) que abra el repo entienda qué es el proyecto, la marca y en qué estado está, **sin depender de memoria externa**.

1. Qué es

Web para el centro de pilates **Reforme Studio Pilates**, en **Florencia, Caquetá (Colombia)**. El encargo es construirles la web.

Requisito clave (no negociable): la web debe **destacar por su UX y UI**, transmitiendo un servicio y sensaciones **premium**, acordes a un estudio de pilates de alta gama. Cada decisión de diseño e interacción debe reforzar esa percepción.

El estudio se trata como **YA ABIERTO**. La web es el punto de entrada donde los usuarios encuentran el estudio y **reservan clases**. (Se eliminó el contador de cuenta atrás a la apertura y toda referencia a "próxima apertura".)

CTA principal: **"Reservar mi clase"** → </registro>.

2. Marca

Paleta de color (HEX):

Uso	Color	HEX	RGB
Principal / marca	Verde selvático	#284435	40, 68, 53
Acento	Dorado / arena	#BE9B69	190, 155, 105
Secundario	Beige claro	#E8E1D9	232, 225, 217
Fondo	Blanco cálido	#F7F6F3	247, 246, 243

La página de tipografía de marca cita un dorado [#C69C6C](#) para textos, pero el acento principal es [#BE9B69](#). La paleta evoca **equilibrio, fluidez y movimiento**.

Tipografía:

- Títulos: **Cormorant** (serif elegante, atemporal; regular + italic).
- Cuerpo: **Lato** (sans-serif).

Identidad:

- Logo/isotipo: figura estilizada tipo persona + ola/montaña. Conceptos: "Movimiento" y "Propósito". Versiones sobre verde, beige, dorado; icono blanco solo sobre fondo oscuro.
- Slogan: **"Movimiento con Propósito"**.

- Pilares: Personalizado · Alta Calidad · Bienestar Integral.

Contacto / ubicación:

- Dirección: Carrera 1 #27-50, Multiplaza Chaira, Florencia.
- Teléfono/WhatsApp: +57 320 9078814.
- Redes (handle [reformestudiopilates](#)): Instagram, Facebook, TikTok, WhatsApp.
- Web de referencia actual: <https://reformestudiopilates.com/>

3. Stack

- **Next.js 16 (App Router) + React 19 + Tailwind CSS v4.**
- TypeScript, estructura con `src/`.
- ⚠ Esta versión de Next tiene breaking changes respecto al conocimiento previo: consultar [node_modules/next/dist/docs/](#) antes de escribir código (ver [AGENTS.md](#)).
- Dependencia añadida para efectos: **lenis** (smooth scroll).
- `package.json` name = `reform-studio-pilates` (carpeta con espacios/mayúsculas). Hay otro `package-lock.json` en el home del usuario → fijado `turbopack.root` en `next.config.ts` (con `path.resolve(__dirname)`, ruta relativa: funciona igual en el Linux de Vercel).

Despliegue

- Repo: github.com/Jhonbic/Reforme-studio-pilates, rama `main`.
- Hospedaje: **Vercel** (plan Hobby). Push a `main` → redespliegue automático.
- ⚠ **Usar siempre la URL de producción:** <https://reform-studio-pilates.vercel.app>
Las otras dos (`...-git-main-...` y la del hash) están tras la *Deployment Protection* de Vercel y **piden iniciar sesión** — en el móvil no hay sesión, así que parecen rotas. Se desactivaría en Settings → Deployment Protection.
- El sitio es 100 % estático (todas las rutas salen o `Static` en el build), por eso no hace falta un contenedor tipo Railway.
- Para ver en el móvil sin desplegar: `npx next dev -H 0.0.0.0` y abrir `http://<IP-del-PC>:3000`.

4. Sistema de diseño

En `src/app/globals.css` vía `@theme`:

- Colores: `verde/dorado/beige/arena` (+ escalas `verde-900/700/500/300`, `dorado-dark/light`).
- Fuentes con `next/font`: Cormorant → `--font-cormorant / font-display / font-serif`; Lato → `font-sans`.
- Utilidades: `.eyebrow`, `.rule-gold`, `.brand-gradient`, animaciones `rise/fade`, `.logo-float` (flotación del logo del hero), sombras `soft/lift`, y CSS de `sheen/ripple/grano`.
- **Colores de datos** (solo panel admin): `--color-chart-1..4` y `--color-estado-ok/aviso/grave`. **No son los de marca** y no se mezclan con ellos — el porqué está en §6, Panel administrativo.

5. Componentes

En `src/components/`:

- **Logo.tsx** — isotipo SVG **recreado a mano**, no el oficial. Lo usan Navbar, Footer y AuthShell. El logo oficial (`public/logo-reforme.png`) solo está en el hero → los dos conviven y **se nota la diferencia de trazo**. Pendiente unificar.
- **Navbar.tsx** — transparente sobre el hero → sólido al hacer scroll. **Menú móvil = drawer del 82% que entra desde la derecha** (elegido por el usuario), con scrim oscurecido, × de cierre explícita, fondo arena claro, enlaces Cormorant y pie con redes + teléfono. Cierra con ×, scrim, **Escape** o al pasar a escritorio.
⚠ **Gotcha resuelto**: el scrim y el drawer van como **hermanos del <header>** (fragmento `<>...</>`), NO dentro de él. El header usa **backdrop-blur** al scrollear, y un **backdrop-filter** convierte al ancestro en bloque contenedor de sus hijos **fixed** → si están dentro, el drawer se encoge/rompe al abrir con la página scrolleada. Mantener fuera.
- **Footer.tsx** — 2 columnas (marca + Visítanos) con las redes reales en círculos de 56px. Se eliminó la columna "Explora": duplicaba la navegación del navbar.
- **Reveal.tsx** — scroll reveal con prop **direction**.
- **Parallax.tsx**.
- **SectionWave.tsx** — ondas orgánicas entre secciones (usa **fill-***).
- **ui/Button.tsx** — sheen + ripple (componente cliente).
- **auth/AuthShell.tsx**, **auth/TextField.tsx**.
- **icons/PilarIcons.tsx** — iconos line-art de los 3 pilares (persona · calendario · corazón), mismos conceptos que la web anterior, trazo alineado con el isotipo.

Panel administrativo en `src/components/admin/` — ver §6:

- **Variacion.tsx** — el indicador «▲ +2,5 %» respecto al periodo anterior. **Un solo sitio decide qué es "bueno"**: no se puede deducir del signo (en cartera pendiente subir es malo). Estaba escrito tres veces, y la tarjeta de tasa de renovación llevaba además la flecha a mano → habría seguido diciendo ▲ aunque el dato bajara. Nunca codifica solo con color: van flecha y signo.
- **Sistema de tarjetas**: **Card.tsx** (tonos **claro** / **oscuro** / **acento**, densidad **normal** / **compacta** / **plana**), **CardHeader.tsx** (emite `<h2>`) y **TablaDeDatos.tsx** (el `<details>` con los datos en tabla). Antes ese puñado de clases estaba copiado a mano en cinco sitios. **ChartCard.tsx** se monta sobre ellos y **conserva su API pública**; re-exporta el tipo **TablaDatos**, que se mudó a **TablaDeDatos.tsx**.
- **secciones.tsx** — las 4 secciones del panel con sus iconos, en un solo sitio. Sin `"use client"` (SVG puro), así lo consumen igual **AdminNav** (cliente) y **AdminTopbar**. Exporta además **SUBSECCIONES**, las rutas hijas con título propio (hoy solo `/admin/usuarios/nuevo`): van **aparte de SECCIONES** para no pintar una quinta pastilla en el menú.
- **admin/campos/** — **el juego de campos de formulario del panel**, nacido con el alta de cliente: **CampoTexto**, **CampoSelect**, **CampoCheck**, **Seccion** y **estilos.ts** (clases + `idsDeCampo()/describedBy()`). ⚠ **No reutiliza `auth/TextField.tsx`**: ese lo usan `/login` y `/registro`, así que tocarlo para

meter select, casillas y `ref` sería tocar la web pública por un formulario del panel — la misma razón por la que `/admin` no usa Navbar ni Footer. Además su estética es la pública (`rounded-xl, bg-white/60`) y aquí manda la del panel (`rounded-full, min-h-[44px]`). Vive en `admin/` y no en `admin/usuarios/` porque Planes y Finanzas van a necesitar formularios.

- `AdminNav.tsx` (cliente, estado activo por `usePathname`), `AdminTopbar.tsx` (cliente, titula la página desde la ruta), `StatTile.tsx`, `TarjetaIngresos.tsx` (la tarjeta héroe), `SeccionPendiente.tsx`.
- `admin/charts/`: `LineChart.tsx`, `GroupedBars.tsx`, `Donut.tsx` (clientes) y `HBars.tsx` (servidor: sin interacción, los valores ya van escritos).

⚠ **Gotcha resuelto:** `LineChart` y `GroupedBars` dibujan en **píxeles reales** (miden el contenedor con `ResizeObserver`, alto fijo por tramo de ancho, fuente de 11px). Con el `viewBox` fijo + `w-full` que tenían antes, el SVG se estiraba con el contenedor: en una tarjeta a ancho completo el factor era $\times 3.9 \rightarrow$ el alto se iba a $\sim 740\text{px}$ y las etiquetas salían a 31px. **No volver a `w-full` sobre un lienzo de tamaño fijo.**

Efectos premium en `src/components/fx/`:

- `SmoothScroll.tsx` — Lenis (inercia + intercepta anclas, offset -80 por navbar).
- `HeroFX.tsx` — canvas único: motas doradas + burbujas que siguen el cursor + ondas "goteo" al clic.
- `Magnetic.tsx` — botones que se imantan al cursor (solo `pointer:fine`).
- Grano/textura global vía `.grain-overlay` (SVG feTurbulence) en el layout.

Todos los efectos respetan `prefers-reduced-motion`. Mantener este principio en cualquier interacción nueva: lo premium nunca debe sacrificar accesibilidad.

6. Páginas

- Landing `src/app/page.tsx`: hero + El estudio + Experiencia (3 pilares) + CTA membresía + Ubicación (mapa embed).
 - **Hero:** logo oficial + eyebrow + titular + párrafo + 2 CTAs, y nada más. Se quitaron los highlights (Grupos reducidos · Reformer premium · Acompañamiento personalizado) porque **repetían los 3 pilares** de más abajo y saturaban el móvil. No reintroducir texto aquí sin quitar otro.
 - **Logo en el hero:** `public/logo-reforme.png` (el oficial, descargado de `reformestudiopilates.com` y recortado a su caja opaca $\rightarrow 554 \times 328$). El PNG viene con el trazo en verde sobre fondo transparente, así que se pinta en blanco con `filter: brightness(0) invert(1)` — igual que hace su web.

⚠ **Gotcha:** ese filtro va **inline**, no con utilidades Tailwind: `filter` es una sola propiedad CSS y `brightness-0 invert` no se combinaba con el `drop-shadow-[...]` arbitrario $\rightarrow$ el logo salía verde sobre verde, invisible. Móvil: centrado encabezando el hero. Escritorio (`lg`): columna derecha, texto a la izquierda. Flota suave con `.logo-float`.

- Se eliminaron del hero los isotipos de marca de agua (**Parallax + Isotype**): con el logo real en primer plano sobraban.
- **3 pilares:** icono en círculo dorado (ver **icons/PilarIcons.tsx**), sin numeración.
- **/login y /registro: solo UI**, sin backend (muestran confirmación simulada). Comparten **auth/AuthShell.tsx**.
 - **/registro valida en cliente** (nombre, email, contraseña ≥8, confirmación, términos). **/login no valida nada:** solo mira el correo para repartir por rol (ver Panel administrativo).
 - Login **sin** "Continuar con Google" (se quitó; la auth real es fase 2 aún sin definir).
 - **AuthShell** lleva los **detalles premium del inicio:** **HeroFX** (estela dorada) en el panel de marca, y en **móvil** una banda de marca verde superior (profundidad radial + HeroFX + logo + titular serif) con **SectionWave** que funde hacia el formulario. Mobile-first.

Panel administrativo (**/admin**)

Fase 3 arrancada. **Solo UI con datos de ejemplo**, sin backend.

- Secciones: **Dashboard** (construido), **Usuarios, Planes, Finanzas** (marcadores con lo previsto en cada una, para que la navegación no dé 404).
- **app/admin/layout.tsx** — armazón propio: barra lateral verde en escritorio, cabecera fija + pastillas con scroll horizontal en móvil. **No usa el Navbar ni el Footer públicos:** son dos productos distintos, y aquí los efectos (grano, parallax, smooth scroll) estorbarían.
 - La lateral es **lg:sticky lg:top-0 lg:h-[100svh]**. ⚠ El **alto explícito no es opcional:** como hijo flex, el **stretch** por defecto la haría tan alta como toda la página y **sticky** no tendría recorrido por el que pegarse — se quedaba quieta y desaparecía al bajar.
 - La columna de contenido lleva **min-w-0**. ⚠ Tampoco es cosmético: un hijo flex tiene **min-width: auto** y no se encoge por debajo de su contenido, así que una rejilla ancha la desbordaría y el **ResizeObserver** de los gráficos mediría un ancho inflado → volvería el bug del estiramiento.
 - **AdminTopbar** titula cada página **desde la ruta**, leyendo el mismo listado (**components/admin/secciones.tsx**) que pinta la navegación: así el menú y el título no se pueden desincronizar. Por eso **SeccionPendiente** ya no lleva **título** — salían dos encabezados iguales, uno encima de otro.
 - **La cuenta (MenuCuenta.tsx) vive en la esquina derecha de esa misma cabecera**, a la altura del título (decisión del usuario). ⚠ **No en una franja propia arriba del todo:** una barra entera para dos datos se come ~64px de alto en todas las pantallas del panel, y en móvil ya hay cabecera de marca y pastillas de navegación por encima. Aquí ocupa un hueco vacío.
 - **Sigue sin haber buscador global ni campana** (el usuario pidió explícitamente quitar el buscador). No hay backend al que buscar ni notificaciones que contar, y un punto rojo sobre una campana que nunca cambia es una mentira pequeña que se paga cuando llegue una de verdad.

- Desplegable con el mismo patrón que **MenuExportar**: **absolute** para no empujar el título, cierre al pulsar fuera y con **Escape**, foco de vuelta al botón. Dentro: nombre, correo, rol, «Ver la web pública» y «Cerrar sesión».
- ⚠ «**Cerrar sesión**» lleva a **/login** pero no cierra nada, y el propio menú lo dice: no hay sesión. Misma regla que el alta de cliente, que tampoco disimula que no guarda.
- El usuario sale de **getUsuarioActual()** (**queries.ts** → **mock.ts**), y se resuelve en el **layout, que es servidor**, bajando como prop: **AdminTopbar** es cliente por **usePathname** y no podría esperar a una versión **async** de esa función el día que haya BD.
- El bloque del título lleva **min-w-0 + truncate**: un título largo se recorta él en vez de empujar el menú fuera de la pantalla.
- **Acceso por rol desde el /login existente** (decisión del usuario, frente a un **/admin/login** aparte). Provisional: el rol se deduce del dominio del correo (**@reforme.com**) porque no hay auth. ⚠ **El panel NO está protegido** hasta que haya autenticación real.

Capa de datos — **src/lib/admin/**:

- **types.ts** — tipos de dominio (importes en COP enteros, sin centavos).
- **mock.ts** — datos de ejemplo. **El único archivo a sustituir** cuando haya BD.
- **queries.ts** — la UI llama SIEMPRE aquí, nunca a **mock.ts**. Al conectar la BD estas funciones pasan a **async** y las pantallas no se tocan.
- **format.ts** — moneda COP con locale fijo **es-CO** (si se dejara al navegador, el HTML del servidor y el del cliente no coincidirían → error de hidratación).
- **catalogos.ts** — listas cerradas del **dominio**: tipos de documento, EPS y **URL_TERMINOS**. **No van en mock.ts** porque **mock.ts** se tira el día que haya BD y estas sobreviven.

Y fuera de **admin/**, porque no es solo del panel:

- **src/lib/validacion.ts** — funciones **puras, sin React**: **esCorreo**, **soloDigitos**, **sinDigitos**, **soloAlfanumerico**, **normalizar**, **claveNombre**, **edad**, **hoyLocalIso**, **normalizarTelefonoPegado**, **esMovilCO**. Aquí dejaron de estar duplicadas la regex de correo (estaba inline en **/registro**) y **normalizar/soloDigitos** (estaban dentro de **PanelUsuarios**).
⚠ **hoyLocalIso()** no usa **toISOString()**: da UTC, y en Colombia (UTC-5) a partir de las 19:00 devolvería ya mañana. Y **edad()** **compara mes y día**, no milisegundos partidos por 365,25, que falla con los bisiestos justo el día del cumpleaños.

Gráficos — **SVG propio, sin dependencias** (**components/admin/charts/**):

LineChart (cruceta + tooltip), **GroupedBars**, **Donut**, **HBars**.

- ⚠ **Gotcha**: los gráficos son componentes de cliente y las páginas de servidor. React **no deja pasar funciones** por esa frontera → no se les pasa un formateador, sino el **nombre** del formato (**FormatoValor**) y lo resuelven

dentro. Mismo motivo por el que **Donut** no acumula ángulos en una variable externa (mutar durante el render lo marca el lint de React).

- Cada gráfico va dentro de **ChartCard**, que incluye **tabla de datos** en un `<details>`. No es opcional: ver la nota de contraste abajo.

Organización del dashboard — rejilla bento. Se reorganizó a partir de una referencia que trajo el usuario, adaptando **solo la organización**: jerarquía por tamaño y contraste en vez de por títulos de sección. Se usan **únicamente los módulos que ya existían** — nada de buscador, campana, feed de actividad ni agenda: no hay datos para eso y no se inventan. (El **avatar sí llegó después**, pero como menú de cuenta en la cabecera, no como bloque del dashboard: ver **MenuCuenta** más arriba.)

- Rejilla única `grid-cols-1 md:grid-cols-6 xl:grid-cols-12`. **12 columnas solo en xl, saltándose lg a propósito**: en **lg** ya está la lateral de 256px y al contenido le quedan ~700px — el tramo más estrecho de todo el escritorio. Doce columnas ahí darían tarjetas de 55px, así que **lg** hereda el reparto de **md**.
- Se **fusionaron** la stat tile "Ingresos del mes" y la tarjeta "Ingresos por mes": contaban el mismo dato dos veces con el mismo peso. Ahora son **TarjetaIngresos**, la tarjeta héroe oscura.
- **Dos tarjetas oscuras** (héroe y tasa de renovación) parten el bloque de blancas. Es lo que da ritmo; con todo blanco, todo pesa igual.
- **Fuera los cuatro <h2> de sección** y el `<header>` de la página (~380px de scroll). El `<h1>` visible lo pone la topbar; en `page.tsx` queda uno `sr-only` para no dejar huérfano el árbol de encabezados, y **CardHeader** emite `<h2>`.
- Ancho a `max-w-[1440px]` (antes `max-w-6xl`). Solo es seguro **porque** los gráficos ya no se estiran.
- **No usar `grid-flow-dense`**: reordena visualmente sin reordenar el DOM y rompe el orden de tabulación.

Contenido actual del Dashboard (tras la poda de jul 2026). Reparto en **xl**:

Fila	Bloques
12	Utilidad del mes · Clientes activos · Cartera por vencer
8 + 4	Ingresos del mes (héroe oscuro) · Ingresos por tipo de plan
4 + 4 + 4	Tasa de renovación (oscura) · Cómo pagan los clientes · Altas y bajas

- **Las dos tarjetas oscuras usan `bg-verde (#284435)`, el verde de marca.** Antes usaban `verde-900 (#1b2f24)`, que es la escala de *profundidad* para hovers y sombras, no un color de superficie: se veía casi negro. El `verde-900` sí se sigue usando **como tinta sobre dorado** (5.47:1); el verde de marca ahí se queda en 4.11:1 y no llega al 4.5 de AA.
- **Efectos del panel: motas + estela, NUNCA goteo.** **HeroFX** acepta `goteo={false}` y así se usa en las dos tarjetas oscuras, en la barra lateral y en la cabecera móvil. El motivo: en el panel casi todo clic va a un control

(un filtro, un desplegable, un enlace) y una onda decorativa encima confunde sobre si la acción se registró. La web pública (landing, `AuthShell`, `Footer`) mantiene el goteo.

- **Las tarjetas encienden el borde en dorado al `hover` y al `focus-within`** (para quien navega con teclado). ⚠ El grosor extra lo pone un **ring, no un `border-2`**: cambiar el ancho del borde en hover desplazaría 1px todo el contenido y se vería como un temblor. Por eso la transición es `transition-[border-color,box-shadow]` y no `transition-colors`.
Las **tres cifras de cabecera** llevan además **sheen**: la estela diagonal `.card-sheen` de la web pública. Son las únicas tarjetas sin tooltip ni contenido que se salga del marco, así que son las únicas donde se puede recortar con `overflow-hidden` sin romper nada.
- ⚠ **Lenis (`SmoothScroll`) está DESACTIVADO en `/admin`**. No es solo criterio: era un **bug**. Lenis cachea la altura desplazable, y los `<details>` de «Ver datos en tabla» hacen crecer la página al abrirse; Lenis no se enteraba y dejaba el tope de scroll en la altura vieja, así que **no se podía bajar a ver la tabla**. Se salta por `usePathname`. En la web pública se le añadió un `ResizeObserver` sobre `body` que llama a `lenis.resize()`, por si aparece algún contenido que cambie de alto.
- **Card con `fx`** monta `HeroFX` (motas, estela y goteo de la web pública) dentro de la tarjeta. Solo en tono **oscuro**: sobre fondo claro el dorado no se ve, y en las tarjetas de datos competiría con el hover de los gráficos.
⚠ El canvas va **detrás del texto con `isolate + -z-10`**, no envolviendo los hijos: envolverlos rompería los `flex-col`, `mt-auto` y `self-start` que las tarjetas reciben desde fuera.
- **Sombras: `--shadow-card`, no `shadow-soft`**. La web pública se queda con `shadow-soft`; el panel tiene su propio token, mucho más ceñido, porque con 8-10 tarjetas a la vez las sombras difusas ensucian la rejilla.
- **Los gráficos atenúan con transición (`.35s var(--ease-smooth)`)**. Sin ella el cambio de opacidad al pasar el ratón se lee como un flash.
- **«Ingresos por mes» filtra por periodo (`GraficaIngresos.tsx`, cliente)**. Se llama "por mes" y no "del mes" porque la tarjeta ya no muestra solo la cifra del mes en curso, sino la serie; el mes al que se refiere el número grande lo dice **detalle** ("Jul 2026 · frente a Jun").
 - Desplegable con **atajos** (12 · 6 · 3 meses) y **rango exacto** (`desde / hasta` con `<select>` agrupados por año con `<optgroup>`).
 - ⚠ **Una sola fuente de verdad: el par [`desde`, `hasta`]**. Los atajos no son un modo aparte, solo escriben ese par — así es imposible que el atajo diga una cosa y el rango explícito otra. La etiqueta del botón muestra siempre lo que se está viendo (12 meses o Ago 25 – Jul 26), nunca "Personalizado".
 - ⚠ **Los rangos imposibles no se validan: no se pueden elegir**. Cada `<select>` solo ofrece los meses que respetan el mínimo de 2 puntos, así que no hay ningún mensaje de error que mostrar ni estado inválido que gestionar.
 - Va **superpuesto (`absolute`)** en la esquina superior derecha de la tarjeta, no en el flujo: abrirlo no puede empujar la gráfica ni estirar la

tarjeta, porque eso descuadraría toda su fila de la rejilla. Cierra al pulsar fuera y con **Escape**. El eyebrow lleva **pr-32** para que en móvil el botón no se le eche encima.

- **<select>** nativo a propósito: en móvil abre el selector del sistema, que es mejor que cualquier lista propia, y trae teclado y accesibilidad gratis.
- La **tabla se filtra con la gráfica** (es su equivalente accesible; si divergieran dejaría de serlo), pero **la cifra grande no cambia**: es siempre el mes en curso frente al anterior.
- **Las tarjetas van SIN descripción a propósito**. El título y el propio gráfico tienen que bastar; si un bloque necesita un párrafo, el problema es el bloque. Excepción deliberada: **Finanzas sí las conserva** — ahí se entra a mirar el dato despacio, no de un vistazo.
- **«Cartera por vencer» NO es la cartera vencida**. No es dinero que ya se debe, sino el importe de renovación de las membresías que vencen en ≤ 7 días (**DIAS_POR_VENCER** en **queries.ts**). El importe y el número de clientes salen de la misma constante a propósito: separarlos haría que dijeran cosas distintas. La cartera vencida real sigue en **getCartera()** / **getTotalCartera()**, sin pintar hoy, reservada para la Cartera detallada de Finanzas.
- **Eliminados del Dashboard**: «Membresías por vencer» (su función la absorbe la cifra de cabecera), «Pagos vencidos por antigüedad» y el aviso de datos de ejemplo. **Movidos a /admin/finanzas**: «Ingresos frente a gastos» y «Gastos por categoría» — son detalle contable, no resumen de negocio.
- **⚠ Ya no hay ningún aviso en pantalla de que las cifras son inventadas**. Se quitó por decisión del usuario. Sigue siendo verdad: todo sale de **mock.ts**.

Usuarios (/admin/usuarios) — construido jul 2026

Listado de personas del estudio. **Solo el listado**: la ficha individual será **/admin/usuarios/[id]**, que **todavía no existe** — las filas ya enlazan ahí, así que hoy dan 404.

- **Dos pestañas, Clientes y Equipo, como estado de cliente y NO como ruta**. Podrían vivir en la URL, pero leer **searchParams** volvería la página dinámica y el panel dejaría de compilar o **Static**. Mismo motivo para búsqueda y filtros. El precio: un filtro concreto no se puede compartir por enlace. Cuando haya backend y paginación de servidor, ese es el momento de subirlos a la URL.
- **Los recuentos viven dentro de las pastillas de filtro** (**Todas 118** • **Activa 87** • **Por vencer 7** • **Vencida 12** • **Inactiva 12**), no en una fila de cifras aparte: el dato y el control por el que se filtra son la misma cosa. Es la misma lógica por la que el dashboard fusionó «Ingresos del mes» con «Ingresos por mes».
- **Cabecera de columnas** (**CabeceraLista.tsx**): **Cliente** • **Plan** • **Estado** • **Vence y Miembro** • **Ro1** • **Estado** • **Desde**. Existe porque la fecha no se entendía — «Vence» y «Desde» son dos cosas distintas y nada lo decía.
 - **Solo en escritorio** (**hidden md:grid**): en móvil la fila se apila y no hay columnas que rotular; ahí cada dato lleva su etiqueta pegada.

- Va **aria-hidden**. Esto **no es una <table>** (la fila entera es un enlace, y una tabla no puede envolver una **<tr>** en un **<a>**), así que no hay relación semántica entre rótulo y celda: un lector de pantalla los leería como una fila de datos sin sentido. Quien navega con lector recibe la información por los **md:sr-only** de cada fila, adosados al dato («Vence 25 jul»). Por eso esos prefijos son **sr-only** y no **hidden**.
- **⚠ rejilla.ts declara la plantilla de columnas en un solo sitio** y la consumen cabecera y filas. Cabecera y fila son rejillas **independientes** — no comparten contexto de tamaño como las celdas de una **<table>** —, así que **ninguna columna puede ser auto**: cada una mediría su propio contenido y los rótulos quedarían desplazados. Todas llevan ancho fijo o fracción, y los fijos salen del contenido más ancho (**2.75rem** = el **Avatar**; **7rem** = la pastilla «**▲** Por vencer»; **5rem** = «25 jul»; **6rem** = «5 feb 2024»). Corolario: **las filas ya no fijan el ancho de su celda de fecha** — lo hace la columna.
- **⚠ La columna «Desde» usa fechaCompacta(), no fecha(iso, true).es-CO** compone la fecha larga como «5 **de** feb **de** 2024», casi el doble de ancho, y era lo que desbordaba la columna. **fechaCompacta** la reconstruye desde **formatToParts** descartando los **literal** — no con **replace(" de ", " ")**, que dependería del orden y del idioma.
- El rótulo **no usa .eyebrow** aunque sea el estilo del panel: su **letter-spacing: 0.32em** desborda las columnas estrechas como «Vence».
- **Filas amplias con avatar (min-h-[72px])**, no tabla densa ni rejilla de tarjetas. La densidad la aporta la paginación (12 por página), no el apretujamiento. **⚠ Un solo DOM para móvil y escritorio**: la fila es un grid que cambia de plantilla (**md:contents** en los bloques internos), no dos versiones con **hidden/md:block** — duplicar el marcado duplicaría el contenido para los lectores de pantalla.
- **La fila entera es un <Link>**, no una fila con un botón dentro: evita interactivos anidados y da un objetivo táctil enorme en móvil.
- **⚠ La tarjeta del listado va con resalte={false}** (decisión del usuario): es la única **Card** del panel que no enciende el borde en dorado al hover. Ocupa casi toda la pantalla y el cursor está siempre dentro, así que no señalaba nada — solo enmarcaba la página entera en dorado. **Card** estrena esa prop, **true** por defecto: el resto del panel no cambia.
- **⚠ La paginación NO pinta números en móvil**, solo **< 4 / 10 >**. El motivo es aritmético, no estético: con 10 páginas son **nueve objetivos táctiles de 44px = ~430px**, y un móvil tiene 375 — se desbordaba con scroll horizontal. No se arregla apretando los botones, porque 44px es el mínimo táctil del panel; se arregla quitando lo que sobra. **No se pierde información**: el «Mostrando 37–48 de 118» de al lado ya dice dónde estás. En **sm** y arriba vuelven los números (**hidden sm:flex**), y ahí van apretados a propósito (**gap-0.5, px-2**): con nueve botones, cada 4px de más son 36px de fila.
- **⚠ La tira de números tiene SIEMPRE el mismo ancho: 7 ranuras fijas (RANURAS en Paginacion.tsx)**. Antes crecía y encogía al cambiar de página

—en la 1 salían 5 huecos (1 2 ... 10) y en la 3 salían 7 (1 2 3 4 ... 10)—, y como la barra alinea a la derecha, los botones **se desplazaban bajo el cursor justo después de pulsar**: se leía como que la barra «se expandía». Cerca de los extremos la ventana no se recorta, se **desplaza**.

- La **elipsis mide lo mismo que un botón** (`min-w-[44px]`). Es la otra mitad del arreglo: con ella más estrecha, cambiar un ... por un número seguiría moviendo la fila aunque el número de ranuras fuera el mismo.
- El «n / N» compacto de móvil lleva `min-w-[76px]` por lo mismo: «1 / 10» y «10 / 10» tienen que ocupar igual o las flechas bailan.
- Los botones de número llevan **border border-transparent siempre** y solo cambian de color al hover: encender un borde que antes no ocupaba sitio movería la tira 1px. Misma regla que las tarjetas y las pestañas.
- **⚠ Se evaluó y se DESCARTÓ un selector de filas por página (12/24/48)** para acortar la tira (decisión del usuario). El problema nunca fue la longitud, sino que la tira cambiaba de tamaño; con las ranuras fijas, un control más que mantener no aporta nada. `POR_PAGINA = 12` sigue siendo una constante.
- **⚠ CLIENTES (en mock.ts) es ahora la fuente de verdad** de lo que ya pintaba el dashboard: `MEMBRESIAS_POR_VENCER` y los dos recuentos de `RESUMEN` **se derivan** de él. Si fueran listas aparte, el dashboard y Usuarios acabarían diciendo cosas distintas. Verificado: el dashboard sigue en 94 activos, 12 inactivos y \$1.400.000 / 5 clientes por vencer.
- **⚠ HOY = "2026-07-25" es una constante**, no `Date.now()`: el panel se renderiza en servidor y los días restantes cambiarían en cada build.
- **⚠ «Activa» en el listado (87) no es «Clientes activos» del dashboard (94).** El dashboard cuenta `Activa + Por vencer` — un cliente que vence el viernes sigue siendo cliente —, mientras la pastilla es el estado exacto para poder filtrar por él, y las cinco tienen que sumar 118. Es deliberado, pero es la única cifra del panel que aparece con dos valores en dos pantallas.
- **EstadoBadge nunca codifica solo con color** — misma doctrina que `Variacion`: cada estado lleva **símbolo + texto** (● `Activa`, ▲ `Por vencer`, ■ `Vencida`, ○ `Inactiva`). Estrena `--color-estado-aviso`, que estaba definido y sin usar. «Inactiva» va en `verde-300` neutro: no es una alarma. Usa `color-mix()` sobre las variables `--color-estado-*` porque la sintaxis `bg-token/10` de Tailwind solo funciona con colores del `@theme`.
- **Se busca por nombre o número de identificación** (decisión del usuario), no por correo: en recepción se identifica a alguien por la cédula. `Cliente` tiene ahora `identificacion`, y **la fila muestra C.C. 1.045.678.912 en vez del correo** — un resultado de búsqueda que no muestra lo que buscaste no se puede verificar de un vistazo. El correo pasa a la ficha `[id]`.
 - **⚠ La cédula se guarda en crudo (solo dígitos) y los puntos los pone documento() al pintar.** Si se guardara formateada, buscar «1045» no encontraría a «1.045.678.912». `documento()` agrupa con regex y **no** con `Intl`: un documento es una cadena de dígitos, no una cantidad — pasarlo por

Number perdería ceros iniciales y chocaría con el límite de precisión en documentos de extranjería más largos.

- **No hay selector de «buscar por»:** si lo escrito contiene dígitos se busca también por cédula, normalizando ambos lados a dígitos («1.045», «1045» y «1 045» son la misma búsqueda). Un modo explícito sería un control más que mantener y que el usuario podría dejar mal puesto.
- La búsqueda del **Equipo** sigue siendo por nombre, correo y rol: no tienen cédula en el mock.
- **Búsqueda sin tildes y sin `debounce`:** normaliza con `NFD + \p{Diacritic}` (quien escribe «Gutierrez» en el móvil espera a «Gutiérrez») y no hay petición que ahorrar — filtra un array en memoria.
- **⚠ Cualquier cambio de filtro vuelve a la página 1**, y la página se acota al vuelo con `Math.min(pagina, totalPaginas)`, **no con un `useEffect`**: así no hay un render intermedio pintando una página que ya no existe.
- **Sin `HeroFX` y sin goteo**, como el resto del panel: es una pantalla de trabajo llena de controles y una onda al clic sembraría duda sobre si la acción se registró.
- **La respuesta al hover de los controles es el dash diagonal dorado (`.control-sheen`), no un relleno de fondo** (decisión del usuario, que descartó el relleno beige que se probó antes). Lo llevan pestañas, pastillas de estado, «Nuevo cliente» y paginación.
 - **⚠ No es `.card-sheen` reutilizado: 0,55 s frente a 1,3 y 0,28 de opacidad frente a 0,22.** Es cuestión de tamaño: una pastilla mide ~100px, y a 1,3 s el barrido se arrastraría mucho después de haber movido el cursor; en tan poco recorrido, en cambio, un dorado más tenue no se llega a ver.
 - **⚠ El dash NO basta como respuesta al hover: es un gesto que pasa y se va, no un estado.** Las pestañas Clientes/Equipo eran solo texto sin borde y no se leían como controles (feedback del usuario: «pasa muy rápido y no se bordea ni se preselecciona»). Ahora la pestaña inactiva lleva **borde permanente** (`border-beige`) que **se enciende en dorado + `ring-2 ring-dorado/25`** al hover y al `focus-visible`, y su texto sube de `verde-300` a `verde-700`. El grosor del borde no cambia nunca —lo que se añade es un `ring`—, por la misma razón que en las tarjetas: pasar a `border-2` movería 1px el texto y se vería como un temblor.
 - **`.control-sheen--lento (1 s)` para los controles GRANDES:** pestañas y botones de cabecera («Exportar», «Nuevo cliente»). La duración se fija con la variable `--sheen-control`, que la clase base deja en 0,55 s. En un control de 120-160px el recorrido es más largo y a 0,55 s el barrido termina antes de que el ojo lo registre; las pastillas y la paginación, más pequeñas, se quedan en 0,55 s.
 - **⚠ El disparador `.control-fx` va en el propio control, no en un `.group` ancestro.** Con el grupo, pasar por *cualquier* pastilla barrería las cinco.
 - **`:hover` y `:focus-visible`, no `:focus`:** al pulsar con el ratón el botón también recibe foco y el barrido saltaría dos veces por un solo clic.

- **No se pinta en los controles que no responden:** ni en el activo (sobre dorado un dorado al 28 % no se ve, y ya está señalado por su relleno) ni en las flechas de paginación deshabilitadas (barrer algo que no hace nada dice lo contrario de lo que pasa).
- **⚠ Los <select> son la excepción y no pueden llevarlo:** un <select> nativo solo admite <option> como hijos, así que no hay dónde colgar el . Se quedan con el borde dorado. No se cambia por un desplegable propio a propósito — ver más arriba por qué el nativo se eligió.
- Apagado por `prefers-reduced-motion`, junto a `.card-sheen` y `.btn-sheen`.
- **⚠ El estado seleccionado es `bg-dorado text-verde-900`, el mismo que la sección activa de AdminNav** (decisión del usuario), en las tres cosas que se pueden seleccionar: pestaña, pastilla de filtro y página actual. Antes era `bg-verde text-arena`.
 - El verde de aquellas pastillas **ya era exactamente el de la lateral** (`bg-verde`, #284435). Se veían distintas porque la lateral lleva `HeroFX` encima (motas doradas + profundidad radial), no por el color.
 - Las tres van juntas: si la página actual siguiera en verde, en la misma pantalla habría dos idiomas para «esto es lo seleccionado».
 - **⚠ El recuento de la pastilla activa pasó de `dorado-light` a `verde-900` sólido.** Sobre dorado, `dorado-light` desaparece; y `verde-900/70` cae a ~3,3:1, por debajo del 4,5 de AA para texto pequeño. Sólido da los **5,47:1** documentados en la nota de accesibilidad. La jerarquía la marca el tamaño.
 - El <aside> y el <header> del panel **siguen en `bg-verde`**: son superficies, no controles.
- **«Exportar» (a CSV) es la única acción que FUNCIONA de verdad** en toda la pantalla, y no necesita backend: los datos ya están en el navegador y el CSV se genera en el cliente (`exportar.ts`). Va a la izquierda de «Nuevo cliente», con su misma forma — son del mismo rango.
 - **Abre un menú con dos casillas, Clientes y Equipo** (`MenuExportar.tsx`, decisión del usuario), y **no depende de la pestaña activa**: estar mirando el Equipo no quiere decir que solo se quiera el Equipo. Antes había que cambiar de pestaña y descargar dos veces. Mismo patrón que el selector de periodo del dashboard: `absolute` para no empujar la fila de pestañas al abrirse, cierre al pulsar fuera y con `Escape`, y el foco vuelve al botón al cerrar.
 - **⚠ Marcar los dos da DOS archivos, no uno.** Clientes y Equipo tienen columnas distintas (plan y vencimiento frente a rol y clases por semana): en la misma hoja, la mitad de las celdas quedarían vacías y no se podría ordenar ni sumar. Son dos tablas. El menú **lo avisa antes** de descargar: dos archivos donde esperabas uno parece un fallo si nadie lo dice.
 - **⚠ Los recuentos van en las etiquetas de las casillas** («Clientes 118»). No es decoración: los filtros de estado y plan **se ocultan** en la pestaña de Equipo pero siguen puestos, así que se puede exportar Clientes con un filtro que no está a la vista. El número es lo que evita la sorpresa.
 - **⚠ Exporta la lista filtrada completa, no la página visible.** Si filtras «Por vencer» esperas los 7, no los 7 de la página 1; y exporta el filtro y no

la base entera porque, si no, los filtros no servirían para sacar datos.

- ⚠ **El separador es ;, no ,.** Excel en español lee la coma como separador decimal: con , el archivo abre entero apelotonado en la columna A.
- ⚠ **El BOM no es opcional.** Sin él, Excel en Windows abre el CSV con la codificación del sistema y «Gutiérrez» sale «GutiÃ©rrez». En este listado los tildes son la mayoría, no un caso raro.
- **Las fechas van en ISO y los importes sin \$ ni comillas:** así Excel los entiende como fecha y como número. Formateados serían texto y no se podrían ordenar ni sumar. Solo se entrecomilla lo que puede romper la fila (" , ;, salto de línea), doblando las comillas internas.
- ⚠ Dos detalles de la descarga, los dos por el caso de **dos archivos seguidos**: el `<a>` se **inserta en el DOM** antes de pulsarlo (uno suelto en memoria no dispara la descarga en todos los navegadores) y `URL.revokeObjectURL` va **diferido 1 s**, no en el mismo tick que el `click()` — revocarlo antes de que el navegador lea el Blob corta la descarga. Sin el revoke, el Blob queda retenido hasta recargar.
- El botón «Descargar» del menú es **disabled** de verdad —no **aria-disabled**— cuando no hay ninguna casilla marcada: ahí no hay ningún porqué que leer, a diferencia del alta.
- Las dos acciones **comparten una sola región** `role="status"`: nunca hay dos avisos a la vez, y con dos regiones un lector anunciaría el anterior otra vez al cambiar el otro.
- El botón «Nuevo cliente» va **arriba a la derecha, en el extremo opuesto de la fila de pestañas** (decisión del usuario): es la única acción de la pantalla y ahí no compete con los filtros. Lleva `aria-disabled="true"`, **no disabled**: el alta no existe (no hay dónde guardarla), y un botón deshabilitado de verdad no recibe foco, así que nadie llegaría a leer el porqué en el `role="status"`. Ese aviso va **en su propia línea** bajo la fila: en la esquina no le queda ancho y al aparecer empujaría las pestañas.
- ⚠ **Card no acepta atributos ARIA arbitrarios** (sus props son `tono`, `densidad`, `fx`, `sheen`, `resalte`, `as`, `className`, `id`). El `role="tabpanel"` va en un `<div>` que la envuelve, en vez de abrir su API por un solo uso.

Alta de cliente (/admin/usuarios/nuevo) — construido jul 2026

Primera pantalla de **captura** de datos del panel. Antes de esto no había ni un formulario en `/admin`, ni utilidad de validación, ni componente de campo que sirviera: además de la página, funda el vocabulario de formularios del panel.

- ⚠ **El formulario NO produce un Cliente, produce una FichaAlta.** Un **Cliente** exige plan, estado, vencimiento e importe, y **el alta no pregunta por el plan** (decisión del usuario: se asigna después). Como además no hay backend ni mutador, nada de esto entra en **CLIENTES**, así que **no hace falta tocar Cliente ni volver sus campos opcionales** — que habría roto la columna de plan

del listado, su filtro, `REPARTO_PLANES` y el CSV. Con la BD, `crearCliente(ficha)` mapeará `ficha` → cliente y ahí se elegirá el plan.

- **La minoría de edad se DERIVA de la fecha de nacimiento, no se pregunta.** No hay casilla «¿es menor?»: el bloque del acudiente aparece solo. Misma doctrina que «los rangos imposibles no se validan, no se pueden elegir».
 - ⚠ `esMenor` es `boolean` | `null`. Mientras no se sepa la fecha de hoy la edad es **desconocida**, no «mayor»: nunca se afirma que alguien es adulto desde un valor que aún no se tiene.
 - ⚠ **Al corregir la fecha de menor a adulto se BORRAN los errores del acudiente.** Si no, quedarían huérfanos y bloquearían el envío desde campos que ya no están en pantalla. Los *valores* se conservan por si la corrección fue el error.
- ⚠ **hoy se obtiene con `useSyncExternalStore`, no en el render ni con `useEffect`.** La ruta es ◦ `Static`: aunque el componente sea de cliente se prerenderiza en el build, así que un `new Date()` en el cuerpo quedaría congelado con la fecha del build → desajuste de hidratación y un `max` de calendario que envejece sin redespargar. Un `useEffect` + `setState` lo arreglaría, pero **el lint de React 19 lo prohíbe** (renders en cascada) y además da un render intermedio.
- ⚠ **`hoyLocalIso()` no usa `toISOString()`.** `toISOString()` da UTC y Colombia va a UTC-5: a partir de las 19:00 devolvería ya mañana, el calendario dejaría elegir el día siguiente y la edad saldría corrida. Se construye con los getters locales. Es el mismo tipo de bug que `format.ts` ya evita con `timeZone: "UTC"`.
- **`edad()` compara mes y día**, no milisegundos partidos por 365,25: con la división, quien cumple años hoy sale de 17 por los bisiestos. La regla es la legal: **cumplir 18 hoy ya es ser mayor de edad.**
- ⚠ **Nunca `type="number"` en documento ni teléfono**, aunque sean numéricos: admite `e/+/-`, **ignora `maxLength`**, pierde ceros iniciales, tiene flechas y devuelve cadena vacía cuando su contenido es inválido. Va `type="text"` + `inputMode="numeric"` + `pattern="\d+"` + **filtrado a dígitos en el `onChange`**. Filtrando, el error «solo números» **no puede llegar a existir**.
 - El filtro **depende del tipo de documento**: un pasaporte lleva letras, así que ahí se filtra alfanumérico en mayúsculas.
 - **Pegar «+57 320 907 8814» funciona:** `normalizarTelefonoPegado` quita el 57 sobrante. Sin eso saldrían 12 dígitos y un error incomprensible justo después de pegar un teléfono correcto.
 - ⚠ **El estado guarda dígitos crudos, sin formato.** Formatear dentro de un input controlado descoloca el cursor al editar por el medio. El formato va en el eco de debajo (`documento()`) y al guardar (`telefonoCO()`).
- **Lo mismo al revés en los NOMBRES: `sinDigitos()` los filtra al teclear**, en los tres campos (cliente, contacto de emergencia y acudiente). Antes solo el nombre del cliente rechazaba números **y solo al enviar**; el del contacto de emergencia no validaba nada y se guardaba con dígitos — era un fallo. Al filtrar, el mensaje «el nombre no lleva números» pasó a ser **código muerto y se borró**: no hay forma de llegar a ese estado.

- **⚠ Documento repetido = ERROR que bloquea. Nombre repetido = AVISO que no bloquea.** No es una incoherencia: dos clientes con la misma cédula son la misma persona metida dos veces, pero **dos personas distintas sí pueden llamarse igual** —en Florencia habrá más de una María Rodríguez—, así que bloquear por nombre impediría dar de alta a alguien real. El aviso dice a quién se parece y deja decidir. La comparación usa `claveNombre()`: sin tildes, sin mayúsculas y con los espacios colapsados, para que «laura gutierrez» y «Laura Gutiérrez» sean la misma.
- **Validación híbrida: «premia pronto, castiga tarde».** `onChange` solo QUITA errores, nunca los pone; `onBlur` solo saca errores de formato **y solo si el campo tiene contenido** (tabular por un campo vacío que ibas a rellenar luego no debe castigarte); el envío valida todo. Una sola función `errorDe()` para los tres momentos, así el mensaje del blur y el del envío no pueden diferir.
- **⚠ Al fallar el envío el foco va al RESUMEN de errores, no al primer campo inválido.** Enfocar el primer campo esconde cuántos problemas hay: arreglas uno, envías, aparece otro —tortura por goteo con catorce campos. `ResumenErrores` es `role="alert" + tabIndex={-1}`, y sus ítems son botones que llevan a cada campo.
- **⚠ `role="alert"` existe UNA sola vez**, en el resumen. Cinco a la vez, uno por campo, son un grito ininteligible en un lector de pantalla; los campos se comunican con `aria-describedby` + `aria-invalid`.
- **`aria-describedby` no existía en NINGÚN sitio del proyecto** antes de esto: el error de `/registro` es un `<p>` suelto sin `id`, que un lector nunca anuncia al enfocar el campo. Lo arregla la convención de `idsDeCampo()` en `components/admin/campos/estilos.ts`, que genera los tres ids en vez de escribirlos a mano.
- **Tercera categoría: avisos ámbar que NO bloquean** (`--color-estado-aviso`). «El teléfono de emergencia es el mismo del cliente» es sospechoso pero legítimo en un menor que usa el móvil de su madre. Bloquear es incorrecto; callar también. El aviso **se calla si hay error**: dos mensajes bajo un mismo campo compiten, y manda el que impide guardar.
- **Los espejos «mismo que el contacto de emergencia» DERIVAN en vivo, no copian una vez.** Copiando, editar después el contacto de emergencia dejaría el dato del acudiente obsoleto en silencio. Solo se guarda el valor *propio*; el efectivo se calcula en cada render. Misma doctrina que el par [`desde`, `hasta`].
 - **⚠ Los campos espejados van `readOnly`, no `disabled`**: un campo deshabilitado sale del orden de tabulación y su valor desaparece del árbol de accesibilidad, así que quien usa lector no podría leer qué se copió.
 - El bloque del acudiente va **después** del contacto de emergencia: los «mismo que...» apuntan hacia arriba y antes no significarían nada.
- **EPS de lista cerrada + «Otra»** (decisión del usuario). En texto libre la base acumularía «Sanitas», «sanitas», «EPS Sanitas» y «SANITAS S.A.» como cuatro EPS distintas y no se podría agrupar nunca. Es el cambio de más valor a largo plazo de esta pantalla, y cuesta un array (`lib/admin/catalogos.ts`).

- **Términos SIEMPRE**, cambiando la redacción según firme el cliente o su acudiente (decisión del usuario, frente a pedirlos solo a los menores): un adulto también debe aceptarlos, y así ningún alta queda sin constancia.
 - «**términos y condiciones**» es un enlace al PDF (`URL_TERMINOS` en `catalogos.ts`), en **azul** por decisión del usuario. ⚠ Es el **único azul de todo el sitio** y rompe la paleta —los demás enlaces sobre claro usan `dorado-dark`—, pero tiene un argumento a favor: `dorado-dark` da 3,81:1 sobre blanco y **no llega al 4,5 de AA**, mientras el `blue-700` da 6,70:1. Si algún día se unifica, lo correcto es llevar el de `/registro` a azul, no este a dorado.
 - ⚠ El `<a>` va **DENTRO** del `<label>` y aun así no marca la casilla. El estándar excluye del comportamiento de la etiqueta los clics sobre contenido interactivo descendiente, así que **no hace falta parar la propagación** —no «arreglarlo» luego.
 - `target="_blank"` y no `download`: así no se pierde un formulario a medio llenar, y desde el visor del PDF se puede guardar igual.
 - ⚠ **El PDF no existe todavía**: el enlace da 404 hasta que se deje el archivo en `public/terminos-y-condiciones.pdf`. No se inventa: es un documento legal del estudio.
- **Contenedor `max-w-5xl`** (1024px). Ni el `max-w-[1440px]` del resto del panel —ahí es ancho porque son tarjetas de datos, y un campo de texto de 1300px es ilegible— ni el `max-w-3xl` que tuvo primero, con el que sobraba tanto margen a los lados que la pantalla parecía vacía. A 1024px los campos emparejados miden ~490px. **Una Card por sección**, no una tarjeta con separadores, porque `Card` ya enciende el borde con `focus-within`: la sección que se edita se resalta sola.
- ⚠ **La rejilla es de 2 columnas pero casi todos los campos ocupan las dos**. Un formulario a dos columnas de verdad rompe el recorrido vertical: al terminar el campo 1 el ojo no sabe si sigue por el 2 o por el 3. Solo se emparejan los campos que son un mismo dato (tipo y número de documento) o que se leen juntos. **La fecha de nacimiento va a media columna**: un calendario ocupa poco y a ancho completo se veía desproporcionado; el hueco de la derecha es donde caben el eco de la edad y la pista del tipo de documento sin apretar nada.
- **`autocomplete="off"` en los campos del cliente**, al revés que en `/registro` y a propósito: allí cada quien escribe sus datos y el autofill ayuda; aquí una recepcionista escribe los de **otra persona** y el navegador le metería los suyos. Excepción: `bday` en la fecha, que no tiene ese riesgo.
- **Al guardar no se guarda nada, y se dice sin eufemismos**. El panel de éxito ofrece **descargar la ficha en CSV** (`csvFicha`), que es la única acción real posible hoy. ⚠ `csvFicha` va en **dos columnas, campo y valor**, y **no añade columnas a `csvClientes`**: una ficha de admisión tiene EPS, acudiente y contacto de emergencia, datos que los 118 clientes no tienen; mezclarlas dejaría 118 filas medio vacías y rompería cualquier plantilla de Excel guardada.
 - **Descartado `sessionStorage`** para que el alta aparezca en el listado: el listado se renderiza en servidor desde `getClientes()`, así que la lista diría

119 y el dashboard 118. Es justo el pecado que `mock.ts` documenta.

- **Descartado un `crearCliente()` que mute el array del módulo:** el estado de módulo es por instancia de servidor, se pierde y se duplica. Peor que no tenerlo, porque *parece* que funciona.
- **Nuevo juego de campos en `components/admin/campos/`, no en `usuarios/`:** Planes y Finanzas van a necesitar formularios. **No se extendió `auth/TextField.tsx`:** lo usan `/login` y `/registro`, así que tocarlo sería tocar la web pública por un formulario del panel — la misma razón por la que `/admin` no usa Navbar ni Footer. Además su estética es la pública (`rounded-xl, bg-white/60`) y aquí manda la del panel.
 - **⚠ Nada de `forwardRef`:** esto es React 19, donde `ref` es una prop normal.
 - El rojo de error sale de `--color-estado-grave`, no del `red-600` de Tailwind que usa la web pública: así el error habla el mismo idioma que las pastillas de estado del listado.
- **SUBSECCIONES en `secciones.tsx`** da título propio a las rutas hijas. Va **fuera de SECCIONES** a propósito: no son entradas de menú y ahí pintarían una quinta pastilla. `esSeccionActiva` sigue marcando «Usuarios» por prefijo, y solo cambia el título. En una subsección el eyebrow deja de ser decorativo y pasa a ser el «← Usuarios»: es la única salida, porque el panel no tiene migas de pan.
- **`normalizar()` y `soloDigitos()` se mudaron** de `PanelUsuarios.tsx` a `src/lib/validacion.ts`, y `/registro` dejó de tener su regex de correo inline.

⚠ shadcn/ui se evaluó y se descartó (jul 2026). Se probó instalar el bloque `@efferd/dashboard-3` en la rama `shadcn-dashboard-3`, ya borrada. Qué se aprendió, por si se vuelve a plantear:

- `shadcn init` **no borra `globals.css`**, añade un `@theme inline` al final — pero ese bloque `redefine --color-chart-1..5` apuntando a grises de croma 0, y como va después, **gana por cascada**: los cuatro gráficos saldrían en escala de grises, justo lo que prohíbe la nota de accesibilidad de este mismo archivo.
- El init también inyecta **Geist** en `layout.tsx` con `variable: '--font-sans'`, que pisa a Lato **en toda la web pública**, no solo en el panel.
- En Windows el sistema de ficheros es *case-insensitive*: el `button.tsx` de shadcn **colisiona con `ui/Button.tsx`**, el botón de marca con sheen y ripple.
- El bloque en sí era un panel de **soporte técnico** (CSAT, tiempo de primera respuesta) y su rejilla era un `sm:grid-cols-2 lg:grid-cols-4` — más simple que este bento. De él solo se aprovecharon dos ideas, reescritas a mano con los tokens de marca: `Variacion` y la lista a sangre.

⚠ Paleta de gráficos ≠ paleta de marca. El verde `#284435` y el dorado `#BE9B69` **fallan** como colores de datos (validado con el verificador de accesibilidad): son tan apagados que se leen como gris y no se distinguen entre sí en daltonismo. Los colores de datos viven en `globals.css` como `--color-chart-1..4` (`#31875C`, `#C68F28`, `#C24A24`, `#8B4A8F`) y pasan las seis comprobaciones sobre fondo arena. Restricciones que arrastran:

- **máximo 4 series** por gráfico (una 5ª se agrupa en "Otros", no se inventa color);
- el dorado queda en 2.64:1 → **siempre etiqueta visible + tabla de datos**;
- se asignan **en orden fijo**, nunca cíclico;
- `--color-estado-*` (ok/aviso/grave) están **reservados** y no se usan como serie.

Matiz para las tarjetas oscuras. Todo lo anterior se midió sobre **fondo arena** y como colores para **distinguir series entre sí**. Sobre `verde-900` y con **una sola serie** no se cumple ninguna de las dos cosas: no hay pares que separar, así que el color no codifica categoría — solo tiene que verse. Ahí sí vale el dorado de marca: `dorado-light` da **7.79:1** sobre verde-900, frente a los **3.21:1** del `chart-1`, que además es verde sobre verde. La tabla de datos se sigue exigiendo. Implementado en la constante `PALETA` de `charts/LineChart.tsx` (prop `tono`), que **solo cambia colores, nunca la geometría**. Los estados tienen su par claro para fondo oscuro: `--color-estado-ok-claro` / `--color-estado-grave-claro`.

Build y lint verificados: `/`, `/login`, `/registro`, `/admin`, `/admin/usuarios`, `/admin/planes`, `/admin/finanzas` — las 7 siguen saliendo o `Static`.

Mobile-first (dispositivo principal de los usuarios)

El móvil es el dispositivo principal → cada cambio se valida primero en móvil. Convenciones ya aplicadas: alturas de hero con `min-h-[100svh]` (no `100vh`, evita el salto por la barra del navegador); objetivos táctiles $\geq 44\text{px}$; CTAs a ancho completo en móvil; todos los efectos respetan `prefers-reduced-motion`.

Todo el contenido va CENTRADO en móvil y pasa a alineación izquierda solo cuando la sección abre a varias columnas (patrón `text-center <bp>:text-left`, con `mx-auto <bp>:mx-0` en los bloques con `max-w-*` y `items-center <bp>:items-start` en los flex). Breakpoint por sección: hero `sm`, tarjetas de pilares y footer `md`, resto `lg`. Excepción deliberada: las **etiquetas y campos de formulario siguen alineados a la izquierda** (legibilidad); solo se centra su encabezado.

7. Fases del proyecto

- **Fase 1 (hecha, jul 2026):** landing + UI de login/registro, sin backend.
- **Fase 2 (pendiente):** auth real (p. ej. Supabase), imágenes reales del estudio.
- **Fase 3 (en curso):** panel administrativo. Dashboard y listado de Usuarios hechos con datos de ejemplo; faltan la ficha `/admin/usuarios/[id]`, Planes y Finanzas.

Ojo al orden: la fase 3 **se adelantó a la 2**. Se está maquetando el panel antes de que exista backend ni autenticación, a propósito, para decidir el diseño con algo delante. Consecuencia directa: **`/admin` está abierto a cualquiera** y todos sus datos son inventados.

Dashboard construido: Estadísticas principales (usuarios, clientes activos, instructores), tarjeta de ingresos del mes con selector de período, reparto por planes, tasa de renovación, métodos de pago, altas y bajas. Rejilla bento con 12 columnas en XL.

Gestión de usuarios construida: Listado de clientes y equipo con pestañas, búsqueda, filtros por estado, paginación (12 por página), exportación a CSV con dos opciones (clientes/equipo). Alta de cliente con formulario validado (nombre, email, teléfono, documento, EPS, acudiente si es menor).

Estructura: Barra lateral fija (verde) en escritorio, cabecera expandible en móvil. Componentes en `src/components/admin/` + datos de ejemplo en `src/lib/admin/mock.ts`.

Diseño: Paleta de marca (verde, dorado, beige), Cormorant + Lato, efectos suaves (motas + estela, sin goteo), mobile-first, dark mode NO (panel de trabajo).

8. Fases del proyecto

- **Fase 1 (landing, jul 2026):** landing + UI de login/registro, sin backend. ☒
- **Fase 1.5 (admin, jul 2026):** panel administrativo (usuarios), mock data. ☒
- **Fase 2 (siguiente):** auth real (Supabase u otro), backend para admin, imágenes reales.
- **Fase 3:** módulos de clases, planes, reportes analíticos.

9. Pendientes inmediatos

Bloqueantes / de riesgo

- ☐ **Proteger `/admin`**. Hoy entra cualquiera escribiendo la URL, y el reparto por rol del login (dominio `@reforme.com`) **no es seguridad**, es un marcador. No desplegar el panel de cara al público hasta resolverlo.
- ☐ Conectar auth real (fase 2). El login ya no ofrece Google; decidir proveedor.

Panel administrativo

- ☐ **Mirar la rejilla bento renderizada** a 375 / 768 / 1280 / 1920. Compila y pasa lint, pero la reorganización **no se ha validado a ojo**. Qué mirar: que no haya scroll horizontal a 1920 (prueba del `min-w-0`), que las alturas de los gráficos sigan siendo 200/240/280 al redimensionar (que no vuelva el estiramiento), la legibilidad del `<details>` en las tarjetas oscuras, y que a 375px la columna única cuente la historia en orden.
- ☒ **Usuarios: listado construido** (clientes + equipo).
- ☒ **Alta de cliente `/admin/usuarios/nuevo`** construida (solo UI: no guarda).
- ☐ **Ficha individual `/admin/usuarios/[id]`**: las filas ya enlazan ahí y hoy dan 404.  Al crearla, que `generateStaticParams` **no emita "nuevo"** (el segmento estático gana igualmente, pero conviene excluirlo).
- ☐ Construir Planes y Finanzas (hoy son marcadores).
- ☐ Sustituir `src/lib/admin/mock.ts` por datos reales cuando haya BD. Ese día, `crearCliente(ficha)` mapea `FichaAlta` → `Cliente` y es donde se asigna el plan (el alta no lo pregunta). Ojo: `mock.ts` congela `HOY = "2026-07-25"` pero el formulario usa la fecha **real**; cuando ambos escriban en el mismo almacén, `diasHasta()` daría valores raros para los clientes nuevos.

Documentos y contenido pendiente de aportar

- ☐ Dejar el PDF de términos y condiciones en `public/terminos-y-condiciones.pdf`. ⚠️ Hasta que esté, el enlace del alta **da 404**. La ruta está en `URL_TERMINOS` (`lib/admin/catalogos.ts`), en un solo sitio. No se redacta desde la web: es un documento legal del estudio.
- ☐ Apuntar ahí también los dos enlaces de `/registro`, que siguen con `href="#"` muertos desde la fase 1 (términos y política de privacidad).

Marca e imagen

- ☒ Logo oficial en el hero (`public/logo-reforme.png`).
- ☒ Panel administrativo fase 1 (usuarios) con mock data.
- ☐ Unificar logo: Navbar, Footer y AuthShell siguen con el isotipo recreado (`Logo.tsx`). Conseguir un **SVG** oficial (el que hay es PNG) + favicon real.
- ☐ Imágenes reales del estudio.
- ☐ Conectar auth real (fase 2, Supabase u otro). El login ya no ofrece Google.
- ☐ Backend para admin (endpoints `/api/usuarios`, conexión a base de datos).
- ☐ Proteger rutas `/admin` con autenticación.

Validación

- ☐ Validar en dispositivo real el drawer móvil y las páginas de auth (capturas pendientes).